Posture Detection

1. Course Content

- Run example programs, use the mediapipe framework to detect human body posture.

2. Run Program

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

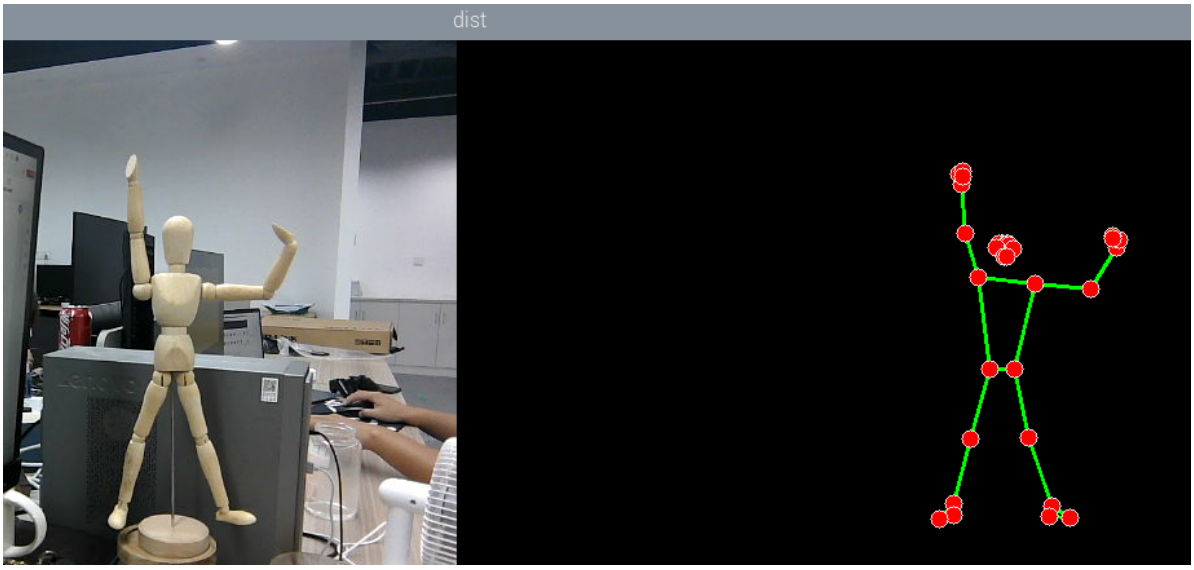
Open a terminal inside the container,

```
exec_m3
```

- Run human body posture detection node

```
ros2 launch m3_bringup mediapipe_demo.launch.py demo:=PoseDetector
```

- After the program runs, as shown in the figure below, the detected posture joint points will be displayed on the right side of the image



3. Source Code Analysis

- Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/mediapipe/02_PoseDetect
or.py
```

Import the required library files,

```
import rclpy
from rclpy.node import Node
#import mediapipe library
import mediapipe as mp
import cv2 as cv
import numpy as np
import time
import os
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2
print("import done")
```

Initialize data and define publishers and subscribers,

```
    def __init__(self, name,mode=False, smooth=True, detectionCon=0.5,
trackCon=0.5):
        super().__init__(name)
        #Parameter declaration and acquisition

 self.declare_parameter('rgb_topic','/ascamera_hp60c/camera_publisher/rgb0/image
')

 rgb_topic=self.get_parameter("rgb_topic").get_parameter_value().string_value

        self.mpPose = mp.solutions.pose
        self.mpDraw = mp.solutions.drawing_utils
        self.pose = self.mpPose.Pose(
            static_image_mode=mode,
            smooth_landmarks=smooth,
            min_detection_confidence=detectionCon,
            min_tracking_confidence=trackCon )
        self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0,
255), thickness=-1, circle_radius=6)
        self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255,
0), thickness=2, circle_radius=2)
        #create a publisher
        self.rgb_bridge = CvBridge()

        self.sub_rgb =
self.create_subscription(Image,rgb_topic,self.get_RGBImageCallBack,10)
```

Color image callback function,

```
def get_RGBImageCallBack(self,msg):
    #Use CvBridge to convert color image message data to image data
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Pass the obtained image to the defined pubPosePoint function, draw=False
means not drawing joint points on the original color image
    frame, img = self.pubPosePoint(rgb_image, draw=False)
    #Merge the two images
    dist = self.frame_combine(frame, img)
    key = cv2.waitKey(1)
    cv.imshow('dist', dist)
```

pubPosePoint function,

```
def pubPosePoint(self, frame, draw=True):
    #Create a new image based on the input image size, the image data type is
uint8
    img = np.zeros(frame.shape, np.uint8)
    #Convert the color space of the input image from BGR to RGB for subsequent
image processing
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing. In
init, the self.pose object was created and initialized
    self.results = self.pose.process(img_RGB)
    #Check if self.results.multi_hand_landmarks exists, i.e., whether posture is
recognized
    if self.results.pose_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
        #Connect each joint point on the previously created blank image
        self.mpDraw.draw_landmarks(img, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    return frame, img
```

The frame_combine image merging function is mentioned in the first lesson of this chapter. You can refer to the analysis of this function in [1. Hand Detection].